

UNIVERSITY OF MACEDONIA

School of Informatics

Department of Applied Informatics

Impact of Matrix Reordering on Sparse Matrix-Vector Multiplication Performance

Bachelor's Thesis by:

Rafail Chrysovalantis Sialakis

Supervisor

Theodoros H. Kaskalis

Professor

Thessaloniki,
July 2026

Acknowledgments

Περίληψη

Abstract

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	1
1.3	Objectives	2
1.4	Contributions	2
1.5	Structure of the Thesis	2
2	Theoretical Background	3
2.1	Introduction to Sparse Matrices	3
2.1.1	Definition	3
2.1.2	Applications	4
2.2	Sparse Matrix-Vector Multiplication	4
2.2.1	Definition	4
2.2.2	Storage Formats	5
2.3	Memory Hierarchy and Cache Behavior	6
2.3.1	Memory Hierarchy	6
2.3.2	Cache Locality	7
2.3.3	Implications for SpMxV Performance	8
2.4	Matrix Reordering	8
2.4.1	Definition	9
2.4.2	Reverse Cuthill-Mckee	9
2.4.3	Approximate Minimum Degree	10
2.4.4	Nested Dissection	11
2.5	Related Work	12
3	Experimental Setup	13
3.1	Hardware Architectures	13
3.1.1	x86 Platform	13
3.1.2	ARM Platform	13
3.1.3	Hardware Comparison	14
3.2	Performance Metrics	14
3.2.1	Floating Point Operations	14
3.2.2	Non-zero Elements per Row	15
3.2.3	Bandwidth	15
3.2.4	Imbalance Ratio	15
3.2.5	Density	16
3.3	Benchmark Matrices	16
3.3.1	Matrix Characteristics	16
3.4	Measurement Methodologies	17
3.4.1	Repeated Ax	17
3.4.2	Input Output Swap	17
3.4.3	Cold Cache	17
3.5	Implementation Details	18
3.5.1	Compiler and Optimization Flags	18
3.5.2	Matrix Loading	18
3.5.3	Thread Configuration	18

3.5.4	Parallel Execution Model	18
3.5.5	Correctness Verification	19
3.6	Execution Modes	19
3.6.1	Batch Execution	19
3.6.2	Cache Evaluation Mode	20
3.6.3	Export of Permutation Vectors	20
3.6.4	Plotting	21
3.7	Technical Limitations	21
4	Results	22
4.1	Overview of Reordering Impact	22
4.1.1	Per-method win/loss summary	22
4.2	Speedup Analysis	22
4.2.1	x86 Results	22
4.2.2	ARM Results	23
4.3	Cache analysis	24
	References	25

List of Figures

1	Sparse matrix pattern for the GHS_psdef/inline_1 matrix from the SuiteSparse Matrix Collection [1].	3
2	A coordinate-form representation of the matrix presented above [2]	5
3	Example of the CSR Storage Format [3].	5
4	An hierarchical view of the computer memory system [4]	7
5	Time difference between processor & memory requests starting from 1980 in logarithmic scale [5]	7
6	The graph before and after applying a permutation of vertices. The two representations differ only in vertex labeling.	9
7	Sparsity pattern of the Cylshell/s3rmt3m3 matrix before and after RCM reordering [1]	10
8	Sparsity pattern of the Cylshell/s3rmt3m3 matrix before and after AMD reordering [1]	11
9	Sparsity pattern of the Cylshell/s3rmt3m3 matrix before and after ND reordering [1]	12
10	Memory hierarchy of the experimental x86 Architecture (x86) platform . .	13
11	Memory hierarchy of the experimental ARM Architecture (ARM) platform	14
12	Load imbalance analysis on Circuit5M, showing a significantly higher workload on T_0 compared to the remaining threads	19
13	<code>make run-all</code> execution pipeline	20
14	<code>make run-all-cache</code> execution pipeline	20
15	<code>make run-perm *.mtx</code> execution pipeline	21
16	<code>make plot</code> execution pipeline	21
17	Per-method win/loss summary across all matrices. A speedup > 1.05 is classified as a win, < 0.95 as a loss, and neutral otherwise.	22
18	Speedup per reordering method on Intel Core i5-1035G1 platform	23
19	Speedup per reordering method on Neoverse N1 platform	24

List of Tables

1	Summary of reordering methods	9
2	Comparison of x86 and ARM experimental platforms	14
3	Characteristics of the sparse matrices used in the experimental evaluation .	17

List of Abbreviations

GFLOPs	Giga Floating Point Operations
FLOPs	Floating Point Operations
SpMxV	Sparse Matrix-Vector Multiplication
PAPI	Performance API
MxM	Matrix-Matrix Multiplication
BFS	Breadth-First Search
COO	Coordinate Format
CSR	Compressed Sparse Row
CPU	Central Processing Unit
FEM	Finite Element Method
CFD	Computational Fluid Dynamics
RAx	Repeated $A \cdot x$
IOs	Input Output swap
x86	x86 Architecture
ARM	ARM Architecture
RCM	Reverse Cuthill-McKee
AMD	Approximate Minimum Degree
ND	Nested Dissection
LU	Lower–Upper
CM	Cuthill-McKee
CG	Conjugate Gradient

1 Introduction

Sparse matrices and sparse matrix computations play a critical role in scientific computing, in a wide variety of problems that belong in the domain of circuit simulation, computational engineering and deep learning. Modern scientific and engineering applications routinely operate on matrices with millions of rows but only a handful of nonzeros per row. Exploiting this sparsity is not merely an optimization, it is a necessity. Storing or operating on these matrices in dense form would be computationally prohibitive, both in terms of memory and execution time.

At the heart of many sparse matrix computations lies the Sparse Matrix-Vector Multiplication (SpMxV) kernel. It is the dominant operation in iterative solvers such as Conjugate Gradient and GMRES, where it may be executed thousands of times before convergence. Despite its conceptual simplicity, SpMxV is notoriously difficult to optimize due to its irregular memory access patterns and its poor use of cache due to the unpredictable locations of nonzero elements. As a result, SpMxV typically achieves only a small fraction of a processor's theoretical peak performance.

One promising approach to address these inefficiencies is matrix reordering, which refers to permuting the rows and columns of a sparse matrix before computation, with the goal of improving data locality, reducing cache misses and balancing the computational load across threads in parallel execution. However, the effectiveness of any given reordering strategy is far from universal. It depends on the structure of the matrix, the characteristics of the target hardware, and the specific performance metric being optimized. This thesis investigates these dependencies systematically, comparing three reordering methods across two hardware architectures.

1.1 Motivation

The motivation for this thesis lies in the growing importance of high-performance sparse computations in modern scientific workloads. As multicore architectures become the norm, the gap between theoretical peak performance and the actual performance of memory-bound kernels such as SpMxV continues to widen. Understanding how structural transformations like reordering can close this gap, and under what conditions, represents both a practically relevant and intellectually challenging problem.

The Sparse Matrix-Vector Multiplication (SpMxV) kernel is a fundamental operation with numerous applications, making its optimization a highly relevant research problem. In this context, a deeper understanding of factors such as matrix structure, cache behavior, and execution efficiency is essential, as improvements in these areas can lead to substantial performance gains in high-performance computing applications.

1.2 Problem Statement

The problem addressed in this thesis is the effect of matrix reordering methods on the performance of the Sparse Matrix-Vector Multiplication kernel. Specifically, this work evaluates three reordering schemes and three performance measurement methodologies. The benchmark is implemented in the C programming language and executed on two different architectures, x86 and ARM, to enable a comparative analysis of the results.

1.3 Objectives

The primary objective of this thesis is to evaluate the impact of matrix reordering strategies on the performance of SpMxV. It investigates whether and how matrix reordering schemes affect load balancing across multi-threaded executions. It further examines how the effectiveness of each reordering depends on matrix characteristics and what are the performance trade-offs. Finally, it concludes on which is the most efficient method on each architecture.

1.4 Contributions

The main contributions of this thesis are a systematic empirical evaluation of three matrix reordering methods on the SpMxV kernel, a cross-architectural comparison of reordering effectiveness on x86 and ARM platforms and an analysis of the relationship between matrix structure, cache behavior, and load balancing under each reordering scheme.

1.5 Structure of the Thesis

This thesis is structured as follows:

1. **Chapter 2:** Introduction to sparse matrices, matrix storage formats, reordering methods, memory & cache hierarchy. Related work on SpMxV performance and optimization techniques using reordering methods is also discussed in this chapter.
2. **Chapter 3:** Description of the experimental setup. This includes the benchmark matrices, the selected measurement methodologies, and the performance metrics used for analysis.
3. **Chapter 4:** Presentation and discussion of the experimental results.
4. **Chapter 5:** Conclusions and practical recommendations that arise from the benchmark results, as well as potential directions for future work.

2 Theoretical Background

This chapter presents the theoretical foundations necessary for understanding the methods and techniques used in this work. More specifically, it introduces the concept of sparse matrices, their properties and their role in various scientific and engineering applications. Additionally, Sparse Matrix-Vector multiplication is discussed, with emphasis on its importance in scientific computing as well as the associated performance bottlenecks. Furthermore, common sparse matrix storage formats are presented, followed by an introduction to matrix reordering techniques, including three widely used methods.

2.1 Introduction to Sparse Matrices

Sparse matrices play a fundamental role in scientific computing and numerical analysis, particularly in applications involving large-scale systems. In this section, we introduce their definition, key characteristics, and typical application domains.

2.1.1 Definition

In the field of numerical analysis and scientific computing, a sparse matrix is defined as a matrix that is filled primarily with zeros as elements of the table [6]. The fraction of zero elements (non-zero elements) in a matrix is called the sparsity (density). A more formal definition of sparsity states that $S\{A\}$ of A is the set of nonzero elements where:

$$S\{A\} = \{(i, j) \mid a_{ij} \neq 0, i, j \leq n\}$$

Operations that depend on dense-matrix algorithms are quite slow and consume huge amounts of memory when applied to large sparse matrices [6]. The number of non-zero elements divided by the total number of elements refers as the density of the matrix.

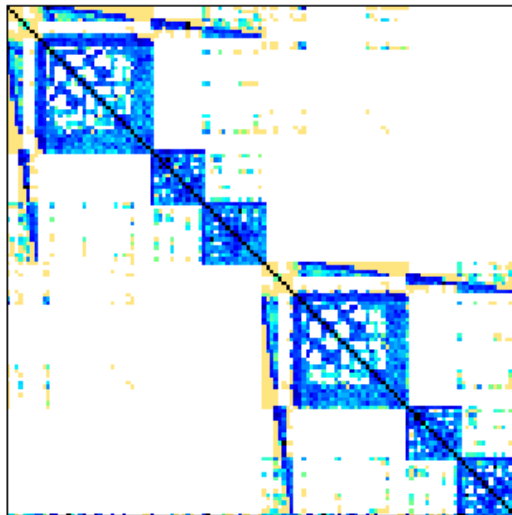


Figure 1: Sparse matrix pattern for the GHS_psdef/inline_1 matrix from the SuiteSparse Matrix Collection [1].

2.1.2 Applications

Sparse matrices arise in a wide range of real-world applications. These include computational fluid dynamics [7, 8], the Finite Element Method and structural problems [9, 10], where matrices are generated from the discretization of physical systems [1]. More specifically, real-world problems are translated into mathematical models with specific constraints, often formulated as partial differential equations [11]. The discretization process often leads to sparse linear systems, for which iterative methods relying on SpMxV are employed [11].

They are also frequently encountered in large scale linear programming problems where algorithms like the revised simplex method operate on large constraint matrices [1]. Other application domains include financial optimization problems, chemical process simulations, semiconductor physics, computer vision, and bioinformatics [1].

Another remarkable application of the SpMxV is the PageRank algorithm introduced by Google [12]. When a search engine returns a list of web pages for a given query, sorting them based on their quality and importance is a hard problem [13]. The PageRank computation falls within the class of numerical linear algebra problems, which depends on the computation of the eigenvector associated with the dominant eigenvalue of the linkage matrix [13]. The power method is the most widely used method, which is dependent on iterative SpMxV routines [13].

Moreover, this operation can be applied in the context of cryptanalysis, where the computation of discrete logarithms in large cyclic groups relies on sparse linear systems [14]. In this context, iterative methods such as Conjugate Gradient (CG) rely heavily on repeated SpMxV computations [14].

The diversity of these application domains highlights the importance of efficient sparse matrix computations in modern scientific and engineering workloads.

2.2 Sparse Matrix-Vector Multiplication

Sparse Matrix-Vector Multiplication is a widely use computational kernel that exists in many scientific applications [15].

2.2.1 Definition

SpMxV is defined as the multiplication of a sparse matrix A with a dense vector x , producing a dense vector y :

$$y = Ax$$

The i -th element of the result vector y is computed as:

$$y_i = \sum_{j=1}^n A_{ij}x_j$$

For sparse matrices, the summation is performed only over the non-zero elements of each row [16]. Sparse matrix-vector operations and more specifically, SpMxV was categorized as one of the “seven dwarfs”, which implies that it belongs in a category with the seven most important numerical methods of at least the next decade [17, 18].

2.2.2 Storage Formats

In this subsection, we present two widely used sparse matrix storage formats, the Coordinate Format (COO) format and the Compressed Sparse Row (CSR) format.

The simplest format to store a sparse matrix, is the COO format, which uses scalars to represent the sizes: (nrows, ncols, nnz), as well as three data arrays: row_ind, col_ind, and values, along with an index parameter. Each tuple, (row_ind[i], col_ind[i], values[i]) is used to represent a non-zero value that belongs in the sparse matrix [19, 20].

Matrices in COO format are not the best option in case of efficiency for sparse matrix operations, although perhaps better than dense matrices. Their purpose is to generate an intermediate representation in order to build other forms of sparse matrix representations, such as CSR [2]. To better illustrate the differences between these formats, we present visual representations of the COO and CSR storage schemes below.

$$\begin{pmatrix} 10 & 0 & 0 & 0 & -2 & 0 \\ 3 & 9 & 0 & 0 & 0 & 3 \\ 0 & 7 & 8 & 7 & 0 & 0 \\ 3 & 0 & 8 & 7 & 5 & 0 \\ 0 & 8 & 0 & 9 & 9 & 13 \\ 0 & 4 & 0 & 0 & 2 & -1 \end{pmatrix}$$

row_ind	0	0	0	1	1	1	2	2	2	2	...	4	4	5	5	5
col_ind	0	1	4	0	1	5	1	2	3	5	...	4	5	1	4	5
val	10	3	-2	3	9	3	7	8	7	3	...	9	13	4	2	-1

Figure 2: A coordinate-form representation of the matrix presented above [2]

The storage format that is most frequently applied for sparse matrices is the Compressed Storage Row (CSR) [17, 21]. The CSR is used for storing the non-zero elements of a mn matrix A using three data arrays to represent its structure. More specifically it uses a floating-point array val that stores the nonzero elements of A , an integer array col_ind that stores the column indices of the elements, and an integer array row_ptr that stores the locations in the array col_ind that start a row in A [2, 19].

$$A = \begin{pmatrix} 5.4 & 1.1 & 0 & 0 & 0 & 0 \\ 0 & 6.3 & 0 & 7.7 & 0 & 8.8 \\ 0 & 0 & 1.1 & 0 & 0 & 0 \\ 0 & 0 & 2.9 & 0 & 3.7 & 2.9 \\ 9.0 & 0 & 0 & 1.1 & 4.5 & 0 \\ 1.1 & 0 & 2.9 & 3.7 & 0 & 1.1 \end{pmatrix}$$

row_ptr : (0 2 5 6 9 12 16)

col_ind : (0 1 1 3 5 2 2 4 5 0 3 4 0 2 3 5)

Figure 3: Example of the CSR Storage Format [3].

The sequential and parallel implementation used for SpMxV following the CSR format are presented below:

Algorithm 1 SpMxV with CSR format

Require: value, col_ind, row_ptr, vector x **Ensure:** vector $y = A \cdot x$ $N \leftarrow$ number of rows**for** $i \leftarrow 0$ to $N - 1$ **do** $y[i] \leftarrow 0$ **for** $j \leftarrow \text{row_ptr}[i]$ to $\text{row_ptr}[i + 1] - 1$ **do** $y[i] \leftarrow y[i] + \text{value}[j] \times x[\text{col_ind}[j]]$ **end for****end for**

Algorithm 2 Parallel SpMxV with CSR format

Require: value, col_ind, row_ptr, vector x **Ensure:** vector $y = A \cdot x$ $N \leftarrow$ number of rows $T \leftarrow$ number of threads**parallel for** $i \leftarrow 0$ to $N - 1$ **do** $\triangleright$ executed in parallel by T threads $y[i] \leftarrow 0$ **for** $j \leftarrow \text{row_ptr}[i]$ to $\text{row_ptr}[i + 1] - 1$ **do** $y[i] \leftarrow y[i] + \text{value}[j] \times x[\text{col_ind}[j]]$ **end for****end parallel for**

In the parallel implementation, it is assumed that the matrix is partitioned by rows, and each thread is assigned $\frac{N}{T}$ rows.

2.3 Memory Hierarchy & Cache Behavior

In the past, it was predicted that the demand for fast memory would increase significantly [5]. Due to the cost of fast memory, memory hierarchy is organized into several levels, where the levels nearest to the processor are the fastest [5]. The goal is a memory system that is both fast and inexpensive [5].

2.3.1 Memory Hierarchy

Memory is designed to balance speed, cost and capacity [22]. At the very top are registers, followed by cache levels (L1, L2, L3), main memory and finally persistent storage [22]. Each level has its purpose: registers store little data but provide the fastest accesses while cache stores copies of data from memory in order to speed-up the access to them [22].

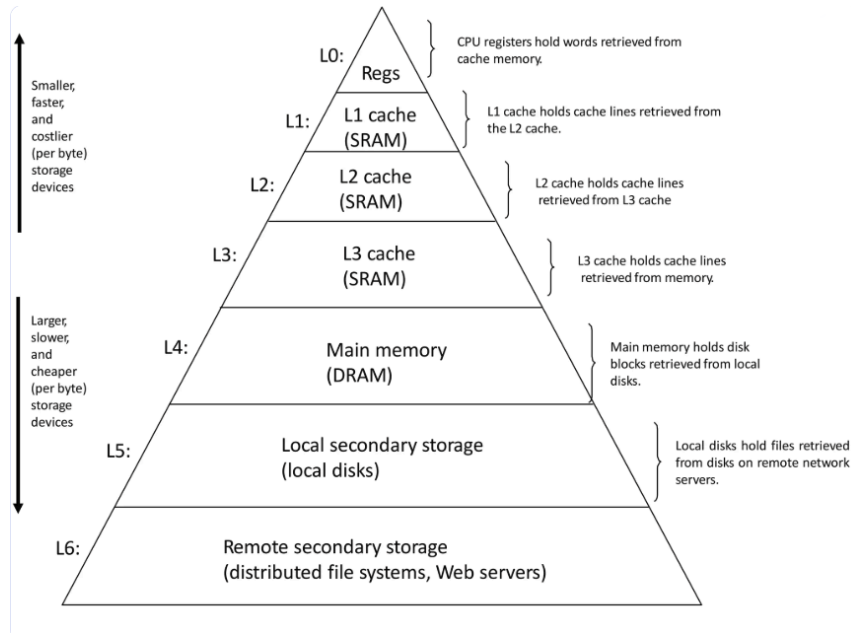


Figure 4: An hierarchical view of the computer memory system [4]

As processors become more powerful, the difference between Central Processing Unit (CPU) speeds and memory accesses increases. This phenomenon is known as the "memory wall" [23]. Multi-core CPUs demand more bandwidth in compare to single-core CPUs [5]. As the number of cores increases, the gap between CPU and memory grows even more [5].

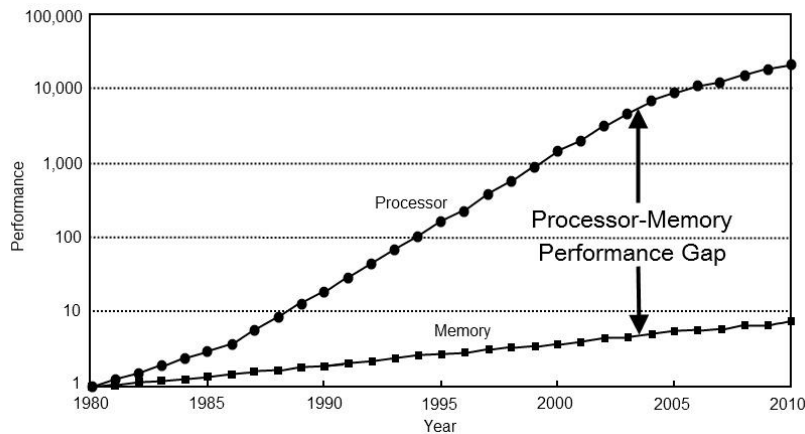


Figure 5: Time difference between processor & memory requests starting from 1980 in logarithmic scale [5]

2.3.2 Cache Locality

Another important concept that will be discussed in the thesis is locality, which is also referred to as the locality of reference [22]. This concept is based on the statement that programs usually contain loops or subroutines, where exist repeated references to a set of instructions [22]. Also instructions that operate on matrices and arrays access memory on a continuous set of data words [22].

The literature states two forms of locality: temporal locality which refers to the bias of a program to reference memory that was referenced in the near past and spatial locality which refers to the bias of a program to reference memory addresses that are near one another [22].

2.3.3 Implications for SpMxV Performance

Improving the performance of SpMxV is quite difficult, because it is affected by the structure of the matrix and its sparsity. As processors become more complex, optimizing SpMxV becomes more challenging [24]. SpMxV is known for its low peak performance which typically achieves less than 20% of theoretical peak performance [25, 26, 27, 28]. These limitations stem from a set of inherent characteristics of the kernel. The literature points that SpMxV presents a set of problems that might have significant effects on its performance [17, 25]. These are listed below:

1. Low memory bandwidth intensity. SpMxV is a kernel that presents very low operational intensity, depending on the input matrix. Poor data reusability and additional metadata accesses increase memory bandwidth demands, resulting in a very low FLOP per byte ratio [25].
2. No temporal locality in the matrix. This occurs unlike other numerical algorithms, such as Matrix-Matrix Multiplication (MxM) and Lower-Upper (LU) factorization, because the elements of the matrix are not being reused [17, 29].
3. Memory cannot be referenced directly. This occurs because only the non-zero elements of the sparse matrix are stored, in order to achieve memory efficiency [17, 30].
4. Poor load balance. This problem appears in matrices that present high variance on non-zero elements per row or varying sparsity patterns across different regions [25]. Depending on how the computation is being distributed among cores, SpMxV may suffer from load-imbalance and underused resources [25]. Some factors that affect it are the sparsity pattern of the input matrix, its storage format, the policy used for parallelism and the resources the hardware platform provides [25].
5. In contrast with dense matrices, where access to the vector x is predictable, in sparse matrices access depends on the sparsity pattern of the matrix [17, 31, 32, 16]. This may result into a significant amount of cache misses, affecting performance [25].

2.4 Matrix Reordering

To begin with, in this section we will define matrix reordering. In this thesis three reordering schemes are going to be evaluated. More specifically the algorithms used are Reverse Cuthill-McKee (RCM), Approximate Minimum Degree (AMD) and Nested Dissection (ND). The considered reordering algorithms assume a structurally symmetric sparsity pattern.

Abbreviation	Method	Description
RCM [33]	Bandwidth reduction	BFS traversal
AMD [34]	Fill-in reduction	Degree-based ordering
ND [35]	Balanced partitioning	Recursive bisection

Table 1: Summary of reordering methods

For unsymmetric matrices, it is common to compute the permutation based on the symmetrized sparsity pattern $A + A^T$ [20]. However this doesn't apply to highly unsymmetric matrices. In such cases a different approach is proposed, named Markowitz pivoting [20]. The case of unsymmetric matrices is not considered in the present work, since all matrices in the benchmark are already structurally symmetric (i.e., symmetric in their sparsity pattern, though not necessarily numerically symmetric).

2.4.1 Definition

Matrix reordering refers to applying row and column permutation in order to alter its properties with an endpoint of optimizing performance [36]. Considering P as the permutation vector (or matrix) that encodes the reordering, the reordered matrix A' can be expressed as:

$$A' = PAP^T$$

The reordering takes place in the graph $G(A)$ that is associated with the adjacency matrix A . More formally the graph $G(A) = (V, E)$, where V is the set of vertices and E is the set of edges. The graph $G(A)$ remains unchanged after the permutation is applied, except the labelling of the vertices [20].

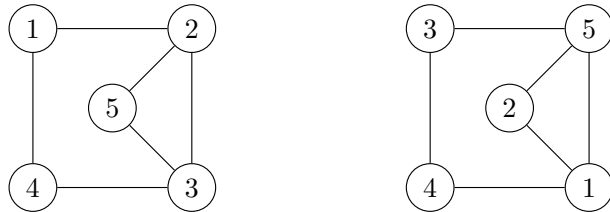


Figure 6: The graph before and after applying a permutation of vertices. The two representations differ only in vertex labeling.

2.4.2 Reverse Cuthill-McKee

First, the notion of bandwidth is introduced. The bandwidth of the matrix is defined as the maximum distance from the main diagonal among all nonzero elements, i.e.,

$$\text{bandwidth}(A) = \max_{a_{ij} \neq 0} |i - j|.$$

Finding a permutation that minimizes the bandwidth of a matrix is a combinatorial optimization problem, and is computationally difficult in general. Therefore heuristic methods are typically employed [20]. The Cuthill-McKee (CM) algorithm is a numerical linear algebra method that was named after Elizabeth Cuthill & James McKee [33]. Considering the graph $G(A)$ induced by the adjacency matrix A , the CM algorithm produces

a permutation P of the vertices of G and is a variant of the Breadth-First Search (BFS) algorithm, where the nodes are sorted by degree in ascending order. Finally RCM is applied by reversing the order of the permutation vector [37, 38].

Algorithm 3 Reverse Cuthill–McKee (RCM) Reordering

Require: adjacency graph $G = (V, E)$

Ensure: permutation vector P of node indices

```

1:  $visited[v] \leftarrow \text{false} \quad \forall v \in V$ 
2:  $P \leftarrow []$  ▷ empty list
3:  $s \leftarrow \text{node with minimum degree}$ 
4:  $Q \leftarrow \text{empty queue}$ 
5:  $enqueue(Q, s)$ 
6:  $visited[s] \leftarrow \text{true}$ 
7: while  $Q \neq \emptyset$  do
8:    $u \leftarrow dequeue(Q)$ 
9:    $append(P, u)$ 
10:   $neighbors \leftarrow \{v \in \text{adj}(u) \mid visited[v] = \text{false}\}$ 
11:  sort  $neighbors$  by increasing degree
12:  for all  $v$  in  $neighbors$  do
13:     $enqueue(Q, v)$ 
14:     $visited[v] \leftarrow \text{true}$ 
15:  end for
16: end while
17:  $P \leftarrow reverse(P)$ 
18: return  $P$ 
    
```

The reordering is applied to the graph $G(A)$ induced by the adjacency matrix A , where $G(A) = (V, E)$. This operation corresponds to a relabelling of the vertices, and therefore does not alter the underlying graph structure, but only the ordering of the vertices [20].

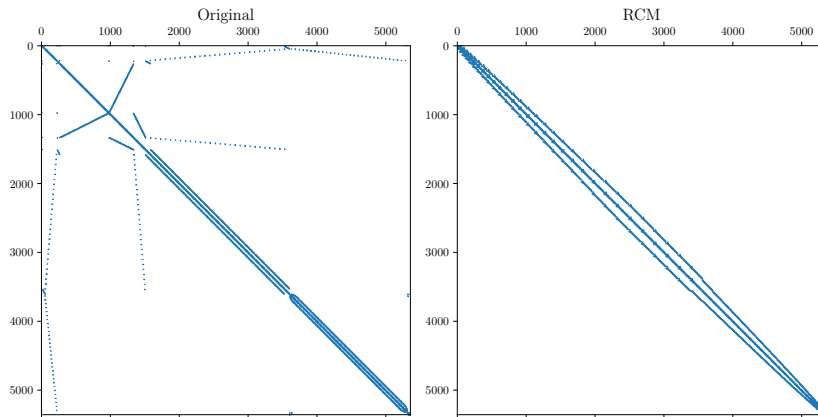


Figure 7: Sparsity pattern of the Cylshell/s3rmt3m3 matrix before and after RCM reordering [1]

2.4.3 Approximate Minimum Degree

First, the notion of fill-in is introduced. Fill-in refers to the appearance of new nonzero entries during sparse matrix factorization that were originally zero in the input matrix.

The Approximate Minimum Degree AMD algorithm, introduced by Patrick R. Amestoy, Timothy A. Davis and Iain S. Duff [34], provides an efficient alternative to classical minimum degree methods. AMD computes the permutation of a sparse matrix prior to factorization, aiming to reduce its fill-in, while lowering computational cost. Instead of maintaining exact degrees, the algorithm uses approximate degree bounds derived from graph-based representations such as the quotient graph, which are inexpensive to compute and often equal to the true degrees [34]. For simplicity, we do not implement the Approximate Minimum Degree (AMD) algorithm ourselves. Instead, we use the `cs_amd` function from the `CSparse` library [39].

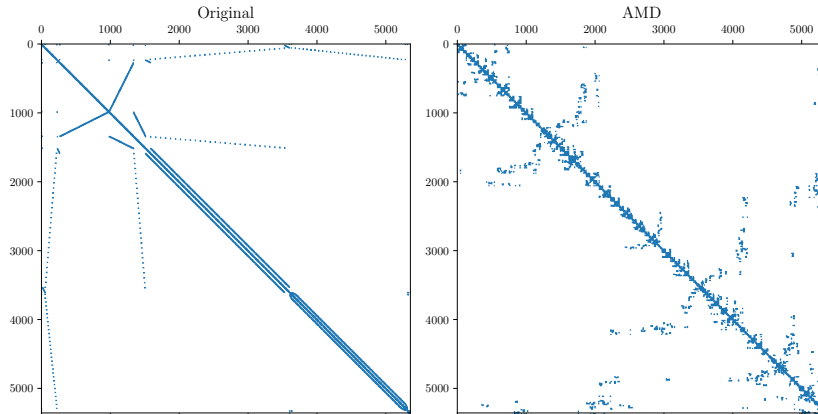


Figure 8: Sparsity pattern of the Cylshell/s3rmt3m3 matrix before and after AMD re-ordering [1]

2.4.4 Nested Dissection

Karypis George and Kumar Vipin stated that there is another class of fill-reducing ordering algorithms, that are referred to as nested dissection (ND) ordering algorithms [35]. These methods recursively bisect the graph associated with the sparse matrix [35]. More specifically, at each level of recursion, the graph is partitioned into nearly equal parts by selecting a vertex separator. The vertices in the separator are then numbered after the vertices in the subgraphs, producing a permutation that reduces fill-in during sparse matrix factorization [35]. In this thesis, we do not implement the Nested Dissection (ND), instead we use the `METIS_NODEND` function from the `METIS` library [40].

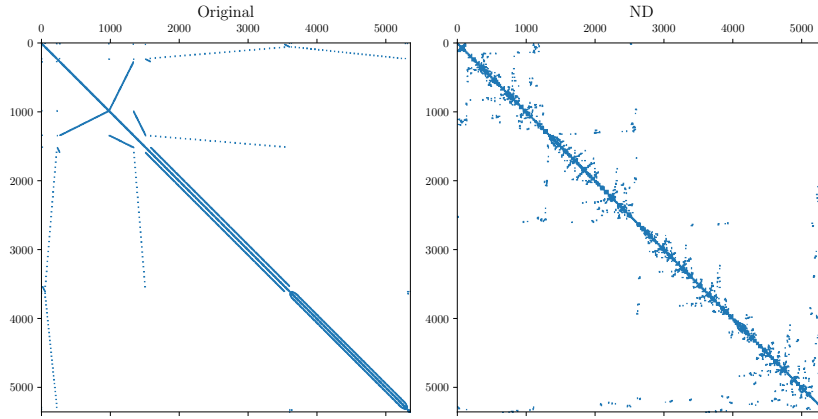


Figure 9: Sparsity pattern of the Cylshell/s3rmt3m3 matrix before and after ND reordering [1]

2.5 Related Work

The impact of matrix reordering on SpMxV performance has been studied extensively in the literature, with varying and sometimes contradictory results.

As reported in [38], early work by Temam and Jalby devised a quantitative model of data locality for SpMxV and discussed the advantages of RCM and ND orderings [41]. Toledo, however, found little or no difference in SpMxV performance when comparing RCM and ND orderings to the original ordering on a small set of matrices [42]. Later improvements of 10-20% were found by Gkountouvas et al. regarding SpMxV performance on multi-core CPUs after the application of RCM reordering [43].

Recently Trotter et al. evaluated six different matrix reordering algorithms on a huge matrix dataset (490 matrices), across 8 multicore architectures and concluded that graph partitioning methods provides the best performance in comparison to other alternatives [38]. Building on this work, Ausdeh et al. examined how the performance of the SpMxV kernel varies over different machines, different reordering strategies and different measurement methodologies and concluded that most reordering schemes deteriorate performance for many matrices, except for RCM [36].

3 Experimental Setup

This section describes the experimental setup used to evaluate the performance of the proposed methods.

3.1 Hardware Architectures

As stated in the above section, the performance of sparse matrix-vector multiplication (SpMxV) is highly dependent on the underlying hardware architecture. Data processing and accessing can be influenced depending on processor design, especially with respect to cache hierarchy, memory bandwidth, and core organization.

In this benchmark, the behavior of the SpMxV kernel is analysed on two distinct architectures: x86 and ARM. In the following subsections, the main characteristics of each platform are presented, with emphasis on their cache hierarchy and memory subsystem.

3.1.1 x86 Platform

The experimental x86 platform is based on an Intel Core i5-1035G1 processor with 4 physical cores and support for 2 threads per core (hyperthreading).

The system features a multi-level cache hierarchy, including 192 KiB of L1 data cache, 128 KiB of L1 instruction cache, 2 MiB of L2 cache, and a shared 6 MiB L3 cache. The processor follows a NUMA-uniform architecture with a single memory node.

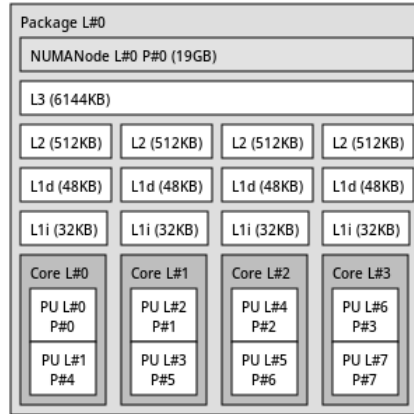


Figure 10: Memory hierarchy of the experimental x86 platform

3.1.2 ARM Platform

The experimental ARM platform is based on an ARM Neoverse-N1 processor with 4 physical cores and no support for simultaneous multithreading. The system operates in 64-bit mode and follows a single-socket, single NUMA-node configuration.

The memory hierarchy consists of 256 KiB of L1 data cache, 256 KiB of L1 instruction cache, 4 MiB of L2 cache, and a shared 32 MiB L3 cache. Compared to the x86 platform, the ARM system provides a significantly larger last-level cache, which can improve data reuse for memory-bound workloads such as SpMxV.

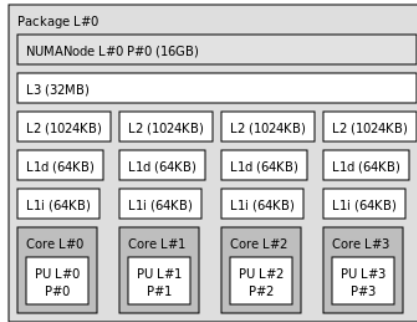


Figure 11: Memory hierarchy of the experimental ARM platform

3.1.3 Hardware Comparison

To facilitate a clear comparison between the two experimental platforms, Table 2 summarizes their main architectural characteristics and differences. These features play a critical role in determining memory efficiency and access behavior.

Feature	Intel i5-1035G1	Neoverse-N1
Architecture	x86_64	aarch64
Cores	4	4
Threads	8 (SMT)	4
L1 Data Cache	192 KiB	256 KiB
L1 Instruction Cache	128 KiB	256 KiB
L2 Cache	2 MiB	4 MiB
L3 Cache	6 MiB	32 MiB
NUMA Nodes	1	1
Frequency	up to 3.6 GHz	NA
Threading Model	2 threads/core	1 thread/core

Table 2: Comparison of x86 and ARM experimental platforms

3.2 Performance Metrics

In order to evaluate the impact of matrix reordering on SpMxV performance, structural metrics are used for each selected matrix, both before and after the application of each reordering methodology. These metrics capture different aspects of matrix structure that are known to influence cache behavior and load balancing during parallel execution. Some of the metrics defined below were chosen based on intuition, while others were chosen based on previous related work [24]

3.2.1 Floating Point Operations

In high-performance computing applications, performance is often measured in terms of Floating Point Operations (FLOPs). Due to the significant increase in computational capability of modern processors, performance is more commonly expressed in Giga Floating Point Operations (GFLOPs). Executing a Sparse Matrix Vector Multiplication, requires two floating point operations: one multiplication and one addition [44]. The execution

rate in GFLOPs/s is defined as:

$$\text{GFLOPs/s} = \frac{2 \cdot nnz}{t \cdot 10^9}$$

where nnz is the number of non-zero elements, t is the execution time in seconds and 10^9 is the conversion factor from FLOPs to GFLOPs.

3.2.2 Non-zero Elements per Row

The average number of non-zero elements per row is defined as:

$$\overline{nnz} = \frac{nnz}{n}$$

where nnz is the total number of non-zero elements and n is the number of rows. In addition, the standard deviation of non-zero elements per row is defined as:

$$\sigma_{nnz} = \sqrt{\frac{1}{n} \sum_{i=1}^n (nnz_i - \overline{nnz})^2}$$

where nnz_i is the number of non-zero elements in row i . A high standard deviation indicates an irregular sparsity pattern, which may lead to load imbalance during parallel execution.

3.2.3 Bandwidth

The bandwidth of a matrix, as formally defined in Section 2.4.2, is used here as a structural metric to assess the effect of each reordering scheme. In addition to the exact bandwidth, the average bandwidth is also computed as:

$$\overline{bw}(A) = \frac{1}{nnz} \sum_{a_{ij} \neq 0} |i - j|$$

which provides a more representative measure of the typical distance of nonzero elements from the diagonal, avoiding the sensitivity of the maximum to outliers.

3.2.4 Imbalance Ratio

The imbalance ratio is a metric that quantifies the degree of load imbalance across threads during parallel SpMxV execution. It is always computed using all available physical cores of each processor. Assuming a row-wise partition of the matrix, where each thread t is assigned a contiguous range of rows, the number of nonzero elements assigned to each thread t is:

$$nnz_t = \text{row_ptr}[\text{row_end}_t] - \text{row_ptr}[\text{row_start}_t]$$

The non-zero elements per thread given perfect load balance are:

$$\overline{nnz}_T = \frac{nnz}{T}$$

where T is the number of threads and nnz is the total number of nonzero elements. The imbalance ratio is then defined as:

$$\text{IR}(A) = \frac{\max_t nnz_t}{\overline{nnz_T}}$$

A value of $\text{IR} = 1$ indicates perfect load balance, while larger values indicate increasing imbalance.

3.2.5 Density

The density of a matrix, as introduced in Section 2.1.1, is formally defined here as the ratio of nonzero elements to the total number of entries in the matrix:

$$\rho(A) = \frac{nnz}{n^2}$$

where n is the number of rows (and columns, since all matrices in this benchmark are square). A lower density indicates a sparser matrix, which typically results in more irregular memory access patterns during SpMxV.

3.3 Benchmark Matrices

The matrices used in the benchmark are selected from different application fields. This can enable a comparative analysis between their structural properties and the performance of the SpMxV. All matrices used in this study are taken from the SuiteSparse Matrix Collection, a widely used repository of real-world sparse matrices [1]. The dataset covers a broad range of application domains including Computational Fluid Dynamics, circuit simulation, the Finite Element Method, linear optimization, and structural analysis. Given the scientific significance of this selection, the benchmark was tailored to reflect realistic engineering workloads.

3.3.1 Matrix Characteristics

The matrices selected for the benchmark and their main characteristics are presented in Table 3. The reported metrics presented are used to capture both the size and sparsity distribution of each matrix, enabling a more comprehensive evaluation of the benchmark behavior across different structural patterns.

Matrix	Category	n	nnz	avg nnz/row	lb	Bandwidth
thermal2	CFD	1,228,045	8,580,313	6.99	1.00	1,226,000
atmosmodd	CFD	1,270,432	8,814,880	6.94	1.00	21,904
nv2	Circuit	1,453,908	52,728,362	36.27	1.25	1,077,636
G3_circuit	Circuit	1,585,478	7,660,826	4.83	1.06	947,128
dgreen	Circuit	1,200,611	38,259,877	31.87	1.26	755,900
circuit5M	Circuit	5,558,326	59,524,291	10.71	2.60	5,556,249
crystk01	FEM	4,875	315,891	64.80	1.03	239
inline_1	FEM	503,712	36,816,342	73.09	1.12	502,403
Flan_1565	FEM	1,564,794	117,406,044	75.03	1.00	20,702
Geo_1438	FEM	1,437,960	63,156,690	43.92	1.00	26,018
Hook_1498	FEM	1,498,023	60,917,445	40.67	1.00	29,036
audikw_1	FEM	943,695	77,651,847	82.28	1.48	925,946
kkt_power	Optimization	2,063,494	14,612,663	7.08	1.32	2,046,911
s3rmt3m3	Structural	5,357	207,695	38.77	1.09	5,302

Table 3: Characteristics of the sparse matrices used in the experimental evaluation

3.4 Measurement Methodologies

The three measurement methodologies used in this benchmark are adopted from Asudeh et al. [36]. Each methodology differs in how the cache state is treated prior to measurement, which can significantly affect the observed performance of the SpMxV kernel. In each measurement we consider the median of `N_RUNS`.

3.4.1 Repeated Ax

Measurements from repeated runs can lower execution times due to caching [36]. While some protocols flush the cache, in real world scenarios, caching is present [36]. In this methodology the benchmark is being executed `WARMUP_RUNS` times, before the measurement is taken. The concept of Repeated A^*x (RAx) is that the execution of

$$y = Ax$$

is repeatedly performed while keeping the operands in memory, simulating a warmer cache state. This results into better cache locality and lower execution times.

3.4.2 Input Output Swap

Asudeh et al. introduced the Input Output swap (IOs) approach in order to avoid artificial caching [36]. This approach proposes swapping the input and output pointers after each SpMxV iteration [36]. This methodology reflects performance more precise because it simulates more realistic cache invalidations [36].

3.4.3 Cold Cache

In this implementation, a `volatile char buffer` is allocated in order to flush the cache. The buffer is larger than all cache levels combined. In order to proceed, after flushing the cache, only one measurement is taken. This simulates a cache that contains garbage in the beginning of the execution, instead of values that must be accessed in order to execute SpMxV efficiently.

3.5 Implementation Details

In this subsection, the implementation details of the benchmark framework are presented. More specifically, the compiler configuration and thread placement policies are discussed, along with the matrix parsing procedure for matrices obtained from the SuiteSparse Matrix Collection [1]. Finally, the validation process used to verify the correctness of the SpMxV computations is described.

3.5.1 Compiler and Optimization Flags

The benchmark was compiled with GCC 15.2 both on x86 and ARM. In both architectures the flag `-O3` is used. The use of identical compiler versions across architectures ensures that any observed performance differences are attributable to hardware characteristics rather than compiler-level optimizations.

3.5.2 Matrix Loading

Each matrix included in the benchmark is saved in Matrix Market format (*.mtx). The parsing process is divided into three stages:

1. Parsing the matrix header
2. Parsing the matrix dimensions and the number of non-zero elements
3. Parsing each non-zero matrix entry and storing it into COO format

The parser can also handle symmetric matrices, where only the diagonal and one triangular part of the matrix are stored in the input file. To simplify subsequent sparse matrix operations, the parser transforms symmetric matrices into an explicit full COO representation by duplicating off-diagonal entries. This preprocessing step increases memory usage but eliminates the need for symmetry-aware kernels during computation.

3.5.3 Thread Configuration

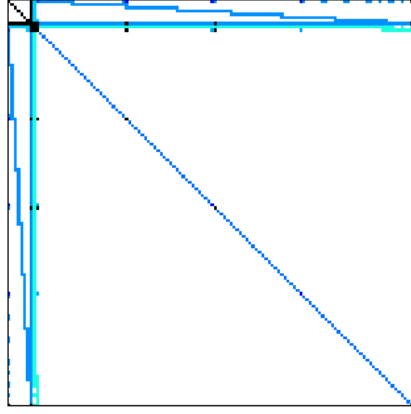
In both systems, four physical cores are available. The Intel i5-1035G1 supports up to eight hardware threads through simultaneous multithreading (SMT), with two threads per core. In contrast, the Neoverse-N1 supports four hardware threads, with one thread per core.

To ensure a fair and reproducible comparison between the two architectures, thread pinning is enforced during the experiments (`OMP_PLACES=cores`, `OMP_PROC_BIND=close`). This prevents thread migration between cores and reduces scheduling-related performance variability.

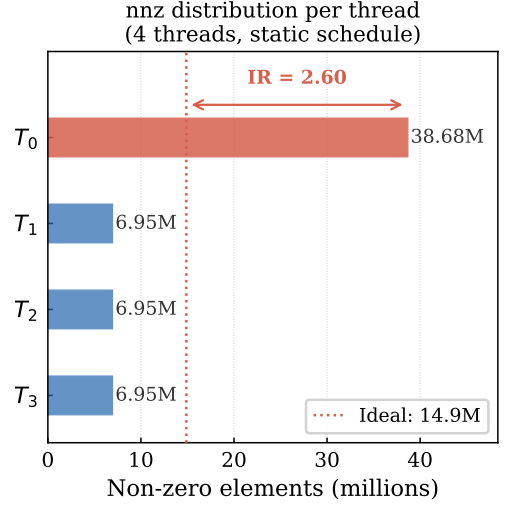
3.5.4 Parallel Execution Model

As stated in Algorithm 2, parallel SpMxV is row-partitioned and implemented using OpenMP in order to ensure deterministic work distribution across threads and comparability with previous studies.

However, this approach may lead to poor load balancing for sparse matrices with irregular sparsity patterns, where the non-zero distribution across rows is highly uneven.



(a) Circuit5M sparsity pattern [1]



(b) Non-zero element distribution

 Figure 12: Load imbalance analysis on Circuit5M, showing a significantly higher workload on T_0 compared to the remaining threads

3.5.5 Correctness Verification

In order to ensure that the reordering in SpMxV produces results consistent with the original (non-reordered) computation, permutation vectors P are used. More specifically, the following equivalence holds:

$$y' = A'x = (PAP^T)(Px) = PA(P^TPx) = PAx = Py,$$

and therefore the original result can be recovered as:

$$y = P^Ty'.$$

This validation is performed before the benchmark begins to guarantee correctness independently of the applied reordering, using a numerical tolerance of $\varepsilon = 1e-10$. Finally the input vector x is initialized as $x_i = (i \bmod 7) + 1$, providing a non-trivial but deterministic test case.

3.6 Execution Modes

In this benchmark three different execution modes are supported. These modes can be defined and selected through the `Makefile` provided in the source code.

3.6.1 Batch Execution

In the batch execution, `time_ms` and GFLOPs are measured using three different measurement methodologies described in Subsection 3.4 and three different reordering schemes described in Subsection 2.4. The measurements are being taken using sequential execution (1 thread) and parallel execution (2,4 threads).

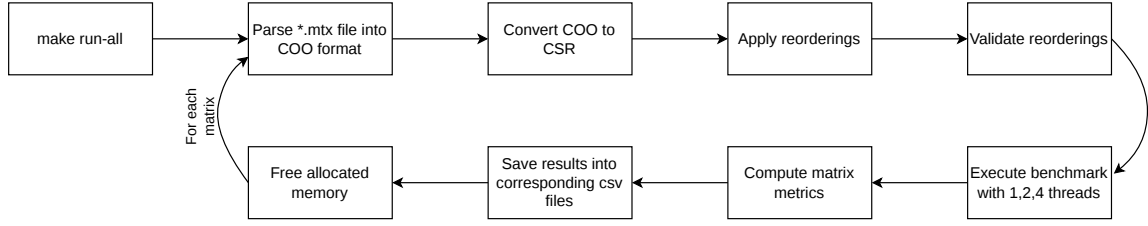


Figure 13: `make run-all` execution pipeline

3.6.2 Cache Evaluation Mode

In the cache evaluation mode, the benchmark is being executed sequentially and only the RAX measurement methodology is taken into account. In order to measure cache related events, PAPI library was utilized.

The Performance API (PAPI) library is used to retrieve hardware performance counter data from modern processors [45]. It provides a portable and simplified interface for performance monitoring, allowing applications developed with PAPI to access processor-specific performance metrics across different platforms without requiring architecture-dependent modifications, as long as PAPI is supported on the target system [45].

The benchmark collects hardware performance counter events related to cache behavior, specifically:

1. PAPI_L1_DCM (Level 1 data cache misses)
2. PAPI_L2_DCM (Level 2 data cache misses)
3. PAPI_L3_TCM (Level 3 total cache misses)

These events are used to evaluate the memory hierarchy efficiency and quantify how often data accesses cannot be served from each cache level, resulting in accesses to lower and slower levels of memory.

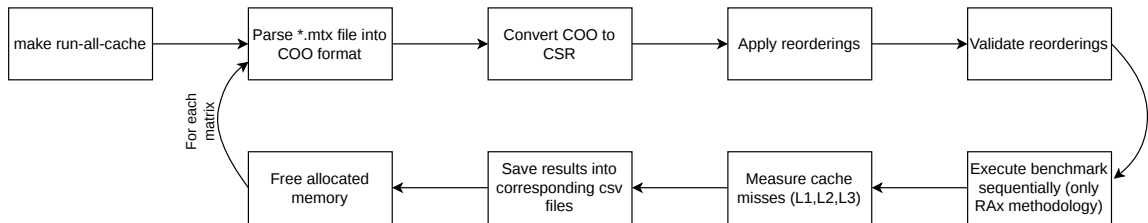


Figure 14: `make run-all-cache` execution pipeline

3.6.3 Export of Permutation Vectors

The export permutation vectors mode is being utilized in order to plot sparse matrices using `pyplot`. This mode does not support batch execution, so it enforces the user to run for one matrix at a time.

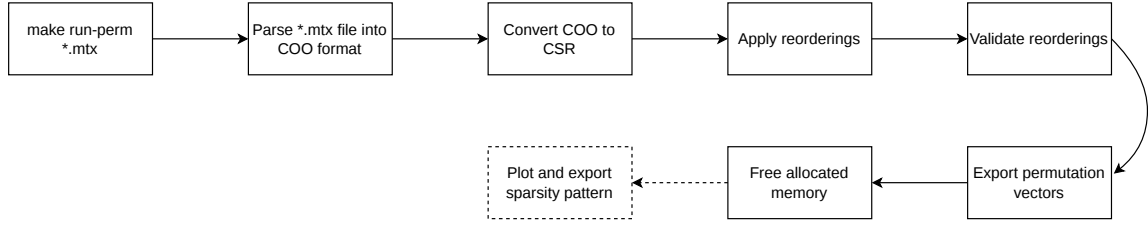


Figure 15: make run-perm *.mtx execution pipeline

3.6.4 Plotting

This execution mode takes place finally after the execution of each benchmark. Its goal is the automated production of figures to achieve a better visualization of the results. More specifically it generates heatmaps, barcharts and tables that will be presented and discussed in detail in Section 4.

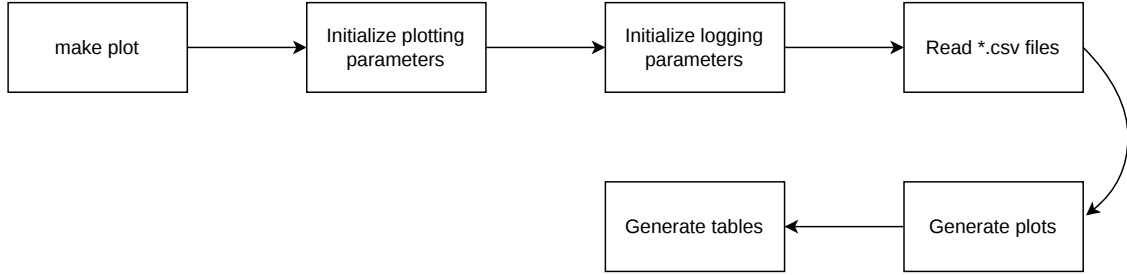


Figure 16: make plot execution pipeline

3.7 Technical Limitations

The x86 platform runs on bare metal while the ARM platform operates within a virtualized Google Cloud environment. This introduces a potential performance bias, as the ARM results may underestimate the true bare metal performance of the Neoverse-N1.

Second, hardware performance counters on the ARM platform were unavailable, so cache miss analysis is presented only for the x86 platform. Despite this limitation, the execution time results on ARM remain valid and the cache behavior can be partially inferred from the structural properties of each reordering and the x86 cache measurements, since the input matrices are identical across both platforms.

4 Results

This section analyzes the results of the benchmark. In particular, it provides a comparative evaluation of the selected matrix reordering methods, the performance measurement methodologies, and the hardware platforms used throughout the study.

4.1 Overview of Reordering Impact

Figure 17 summarizes the overall impact of each reordering method across all benchmark matrices on both platforms. A matrix is classified as a *win* if the reordering achieves a speedup greater than 1.05, a *loss* if it falls below 0.95, and *neutral* otherwise.

4.1.1 Per-method win/loss summary

As shown in Figure 17, reordering benefits depend highly on the structure of the matrix, so no single method dominates across all cases. On both platforms, RCM achieves the highest number of wins, consistent with the findings of Asudeh et al. [36]. AMD and ND, while effective for specific matrices, produce a higher number of slowdowns due to their fill-reducing nature, which does not necessarily improve data locality for SpMxV.

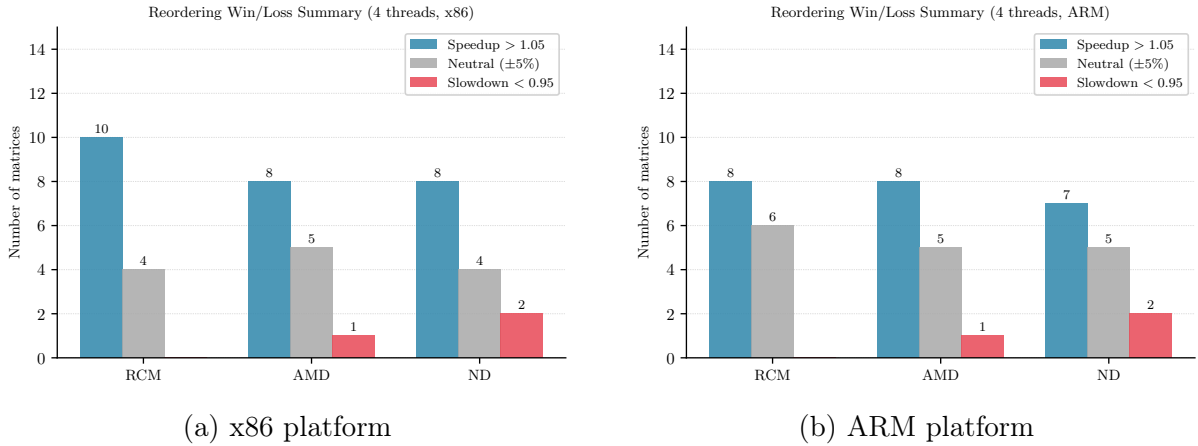


Figure 17: Per-method win/loss summary across all matrices. A speedup > 1.05 is classified as a win, < 0.95 as a loss, and neutral otherwise.

4.2 Speedup Analysis

In this subsection, speedup results are presented for both the x86 and ARM platforms. Figures 18 and 19 present speedup heatmaps for each matrix and reordering method used in the benchmark, measured at 4 threads using the RAx methodology. Each cell represents the speedup of the reordered execution relative to the original (unreordered) baseline, where values greater than 1.0 indicate a performance improvement.

4.2.1 x86 Results

Figure 18 presents the speedup heatmap for the x86 platform using 4 threads. It can be obvious that in some cases the SpMxV performance can be benefited, while in other cases it may be negatively affected.

It is evident that nv2 achieves impressive speedups across all reordering methods, with AMD delivering the highest gains, followed by RCM, and then ND. This may be interpreted by its average bandwidth without applying any orderings which reaches up to 420015.41. Each method provides a significant reduction on this metric, with RCM achieving a 77.89% reduction, followed by AMD with 47.81%, and ND with 34.31%. Also the Imbalance Ratio (IR) metric reaches 1.25 with no reordering and presents a reduction of 19.20% both on AMD and RCM and 18.40% on ND.

The dgreen matrix exhibits characteristics similar to nv2, however, the observed speedups differ significantly. This will be further discussed in the following Subsection 4.3, due to the fact that the metrics alone may not fully capture the underlying behavior.

On the other side Circuit5M is a very imbalanced matrix as shown in Figure 12. It presents a speedup in both AMD and RCM but a dramatic slowdown on ND. This may be explained due to the increase of its average bandwidth by 6.14%. In addition to that the imbalance ratio factor presented a reduction of 18.85% on ND, which is the least of the three reorderings, 38.85% on RCM and finally 40.38% on AMD.

At the lower end of the spectrum, matrices such as Flan_1565, Hook_1498, and Geo_1438 show little to no benefit from reordering on both platforms, with speedups near or slightly below 1.0. This indicates that these matrices likely already exhibit favorable memory access patterns in their natural ordering, or that their structure does not lend itself to the locality improvements that reordering methods target.



Figure 18: Speedup per reordering method on Intel Core i5-1035G1 platform

4.2.2 ARM Results

Figure 19 presents the speedup heatmap for the ARM platform using 4 threads. The overall pattern is consistent with the x86 results, with the same matrices standing out as notable cases; however, the ARM platform generally exhibits higher speedups across

most matrices and reordering methods.



Figure 19: Speedup per reordering method on Neoverse N1 platform

4.3 Cache analysis

References

- [1] T. A. Davis and Y. Hu, “The university of florida sparse matrix collection,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 38, no. 1, pp. 1:1–1:25, 2011.
- [2] A. W. Appel, “Formal verification of coo to csr sparse matrix conversion (invited paper),” in *Proceedings of the International Workshop on Verification of Scientific Software (VSS 2025)*, ser. EPTCS, S. F. Siegel and G. Gopalakrishnan, Eds., vol. 432. Electronic Proceedings in Theoretical Computer Science, 2025, pp. 3–18, invited paper.
- [3] K. Kourtis, G. Goumas, and N. Koziris, “Optimizing sparse matrix-vector multiplication using index and value compression,” 05 2008, pp. 87–96.
- [4] A. Irfan, “Computer memory hierarchy,” <https://www.slideshare.net/slideshow/computer-memory-hierarchy-computer-architecture/127332146#5>, accessed: 2026.
- [5] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.
- [6] D. Yan, T. Wu, Y. Liu, and Y. Gao, “An efficient sparse-dense matrix multiplication on a multicore system,” in *2017 IEEE 17th International Conference on Communication Technology (ICCT)*. IEEE, 2017, pp. 1880–1883.
- [7] G. Oyarzun, D. Peyrolon, C. Alvarez, and X. Martorell, “An fpga cached sparse matrix vector product (spmv) for unstructured computational fluid dynamics simulations,” *arXiv preprint arXiv:2107.12371*, 2021.
- [8] X. Álvarez-Farré, A. Gorobets, and F. X. Trias, “A hierarchical parallel implementation for heterogeneous computing. application to algebra-based cfd simulations on hybrid supercomputers,” *Computers & Fluids*, vol. 214, p. 104768, 2021.
- [9] A. Altinkaynak, “An efficient sparse matrix-vector multiplication on cuda-enabled graphic processing units for finite element method simulations,” *International Journal for Numerical Methods in Engineering*, vol. 110, no. 1, pp. 57–78, 2017.
- [10] P. Grigoraş, P. Burovskiy, W. Luk, and S. Sherwin, “Optimising sparse matrix vector multiplication for large scale fem problems on fpga,” in *2016 26th international conference on field programmable logic and applications (FPL)*. IEEE, 2016, pp. 1–9.
- [11] J. Gao, B. Liu, W. Ji, and H. Huang, “A systematic literature survey of sparse matrix-vector multiplication. arxiv abs/2404.06047 (2024),” 2024.
- [12] L. Page, S. Brin, R. Motwani, and T. Winograd, “The pagerank citation ranking : Bringing order to the web,” in *The Web Conference*, 1999. [Online]. Available: <https://api.semanticscholar.org/CorpusID:1508503>
- [13] T. Wu, B. Wang, Y. Shan, F. Yan, Y. Wang, and N. Xu, “Efficient pagerank and spmv computation on amd gpus,” in *2010 39th International Conference on Parallel Processing*. IEEE, 2010, pp. 81–89.

- [14] H. Jeljeli, “Accelerating iterative spmv for the discrete logarithm problem using gpus,” in *International Workshop on the Arithmetic of Finite Fields*. Springer, 2014, pp. 25–44.
- [15] K. Akbudak, E. Kayaaslan, and C. Aykanat, “Hypergraph partitioning based models and methods for exploiting cache locality in sparse matrix-vector multiplication,” *SIAM Journal on Scientific Computing*, vol. 35, no. 3, pp. C237–C262, 2013.
- [16] E.-J. Im, “Optimizing the performance of sparse matrix-vector multiplication,” Ph.D. dissertation, EECS Department, University of California, Berkeley, 2000. [Online]. Available: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2000/CSD-00-1104.pdf>
- [17] G. Goumas, K. Kourtis, N. Anastopoulos, V. Karakasis, and N. Koziris, “Performance evaluation of the sparse matrix-vector multiplication on modern architectures,” in *Proceedings of the International Conference on Parallel Processing (ICPP)*, 2008, pp. 273–282.
- [18] K. Asanovic, R. Bodik, B. C. Catanzaro, J. J. Gebis, P. Husbands, K. Keutzer, D. A. Patterson, W. L. Plishker, J. Shalf, S. W. Williams, and K. A. Yelick, “The landscape of parallel computing research: A view from berkeley,” EECS Department, University of California, Berkeley, Technical Report UCB/EECS-2006-183, Dec. 2006.
- [19] I. Corporation, “Sparse matrix storage formats in onemkl,” <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-dpcpp/2025-2/sparse-storage-formats.html>, 2025, accessed: 2026-03-25.
- [20] J. Scott and M. Tũma, *Sparse Matrix Ordering Algorithms*. Cham: Springer International Publishing, 2023, pp. 135–161. [Online]. Available: https://doi.org/10.1007/978-3-031-25820-6_8
- [21] R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine, and H. V. der Vorst, *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods, 2nd Edition*. Philadelphia, PA: SIAM, 1994.
- [22] W. Stallings, *Computer organization and architecture: designing for performance*. Pearson Education India, 2016.
- [23] W. A. Wulf and S. A. McKee, “Hitting the memory wall: Implications of the obvious,” *ACM SIGARCH computer architecture news*, vol. 23, no. 1, pp. 20–24, 1995.
- [24] S. Chen, J. Fang, D. Chen, C. Xu, and Z. Wang, “Optimizing sparse matrix-vector multiplication on emerging many-core architectures,” *arXiv preprint arXiv:1805.11938*, 2018.
- [25] P. Mpakos, D. Galanopoulos, P. Anastasiadis, N. Papadopoulou, N. Koziris, and G. Goumas, “Feature-based spmv performance analysis on contemporary devices,” in *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2023, pp. 668–679.
- [26] G. Tan, J. Liu, and J. Li, “Design and implementation of adaptive spmv library for multicore and many-core architecture,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 44, no. 4, pp. 1–25, 2018.

- [27] S. Filippone, V. Cardellini, D. Barbieri, and A. Fanfarillo, “Sparse matrix-vector multiplication on gpgpus,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 43, no. 4, pp. 1–49, 2017.
- [28] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel, “Optimization of sparse matrix-vector multiplication on emerging multicore platforms,” in *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*, 2007, pp. 1–12.
- [29] A. Buttari, V. Eijkhout, J. Langou, and S. Filippone, “Performance optimization and modeling of blocked sparse kernels,” *IJHPCA*, vol. 21, pp. 467–484, 11 2007.
- [30] A. Pinar and M. T. Heath, “Improving performance of sparse matrix-vector multiplication,” in *Proceedings of the 1999 ACM/IEEE Conference on Supercomputing (SC ’99)*. Portland, OR, USA: ACM SIGARCH and IEEE, November 1999. [Online]. Available: <https://bebop.cs.berkeley.edu/readings/pinar99-reord.pdf>
- [31] R. Geus and S. Röllin, “Towards a fast parallel sparse symmetric matrix-vector multiplication,” *Parallel Computing*, vol. 27, no. 7, pp. 883–896, 2001, linear systems and associated problems. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167819101000734>
- [32] J. Pichel, D. B. Heras, J. Cabaleiro, and F. Rivera, “Improving the locality of the sparse matrix-vector product on shared memory multiprocessors,” *16th Euromicro Conference on Parallel, Distributed and Network-Based Processing (PDP 2008)*, vol. 0, p. 66, 01 2004.
- [33] E. Cuthill and J. McKee, “Reducing the bandwidth of sparse symmetric matrices,” in *Proceedings of the 1969 24th National Conference*, ser. ACM ’69. New York, NY, USA: ACM, 1969, pp. 157–172. [Online]. Available: <https://doi.org/10.1145/800195.805928>
- [34] P. R. Amestoy, T. A. Davis, and I. S. Duff, “An approximate minimum degree ordering algorithm,” *SIAM Journal on Matrix Analysis and Applications*, vol. 17, no. 4, pp. 886–905, 1996. [Online]. Available: <https://doi.org/10.1137/S0895479894278952>
- [35] G. Karypis and V. Kumar, “A fast and high quality multilevel scheme for partitioning irregular graphs,” *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998. [Online]. Available: <https://doi.org/10.1137/S1064827595287997>
- [36] O. Asudeh, S. Mahdipour Saravani, F. Rastello, G. Sabin, and P. Sadayappan, “How effective is matrix reordering for improving performance of sparse matrix-vector multiplication?” in *Proceedings of the SC’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 823–827.
- [37] W.-H. Liu and A. H. Sherman, “Comparative analysis of the cuthill–mckee and the reverse cuthill–mckee ordering algorithms for sparse matrices,” *SIAM Journal on Numerical Analysis*, vol. 13, no. 2, pp. 198–213, 1976.

- [38] J. D. Trotter, S. Ekmekçi, J. Langguth, T. Torun, E. Düzakın, A. Ilic, and D. Unat, “Bringing order to sparsity: A sparse matrix reordering study on multicore cpus,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23)*. ACM, 2023. [Online]. Available: <https://doi.org/10.1145/3581784.3607046>
- [39] T. A. Davis, “Csparse: a concise sparse matrix package,” <http://faculty.cse.tamu.edu/davis/suitesparse.html>, accessed: 2026-03-28.
- [40] G. Karypis and V. Kumar, *METIS: A Software Package for Partitioning Unstructured Graphs, Partitioning Meshes, and Computing Fill-Reducing Orderings of Sparse Matrices*, September 1998.
- [41] O. Temam and W. Jalby, “Characterizing the behavior of sparse algorithms on caches,” Ph.D. dissertation, INRIA, 1992.
- [42] S. Toledo, “Improving the memory-system performance of sparse-matrix vector multiplication,” *IBM Journal of research and development*, vol. 41, no. 6, pp. 711–725, 1997.
- [43] T. Gkountouvas, V. Karakasis, K. Kourtis, G. Goumas, and N. Koziris, “Improving the performance of the symmetric sparse matrix-vector multiplication in multicore,” in *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*. IEEE, 2013, pp. 273–283.
- [44] E. Montagne and E. Aymerich, “Let’s agree on computing flops for the symmetric sparse matrix vector product,” in *Proceedings of the 24th High Performance Computing Symposium*, 2016, pp. 1–6.
- [45] F. Gorsler, R. Berrendorf, and D.-I. C. Iwainsky, “Development of a papi backend for the sun niagara 2 processor,” 2009.